

RIOT: Unleashing Multi-core OCaml with Erlang-style Parallelism

Leandro Ostera
leandro@abstractmachines.dev
Abstract Machines
Stockholm, Sweden

Abstract

We present RIOT, bringing Erlang/OTP’s concurrency to OCaml via effect handlers. Our key insight: effect handlers provide the perfect abstraction for lightweight processes, enabling industrial-strength actor concurrency without complex runtime machinery. RIOT achieves process isolation, symmetric multiprocessing, and supervision trees—previously requiring purpose-built VMs.

1 Introduction

Erlang/OTP sets the gold standard for concurrent systems through failure isolation, symmetric multiprocessing, and supervision trees. Previous actor implementations in mainstream languages compromise on these essentials, using OS threads or monadic styles that miss Erlang’s elegance.

OCaml 5’s effect handlers fundamentally change this. They provide direct-style programming with efficient suspension/resumption—exactly what’s needed for lightweight processes. RIOT leverages this to bring Erlang’s full model to OCaml.

Contributions: (1) Effect-based lightweight processes eliminating complex continuation management, (2) Type-safe messaging via extensible variants, (3) Selective receive through effects, (4) Production features: timing wheels, supervision, SMP.

2 Effect Handlers as the Enabler

The core innovation—using effect handlers for process management:

```
type _ Effect.t +=  
  | Yield : unit Effect.t  
  | Receive : 'msg selector -> 'msg Effect.t  
  
(* Process captures continuation on effect *)  
match proc_state with  
| Suspended (k, Receive selector) ->  
  handle_receive k selector
```

This elegantly implements cooperative multitasking. When receiving, the handler captures the continuation, enabling process switching without manual CPS transformation.

Selective Receive—Erlang’s killer feature via save queues:

```
match selector msg with  
| `select v -> Continue v  
| `skip -> save_for_later msg
```

Type-Safe Messages via extensible variants:

```
type Message.t = ..  
type Message.t += Request of data | Response of result
```

3 Architecture

Three-tier design: (1) MiniRiot: single-core cooperative scheduler, (2) Multi-core work-stealing, (3) Distributed actors. Core components:

Scheduler: Process tables, per-core run queues, work-stealing for load balance. **Processes:** 800 bytes each, isolated state, mailbox + save queue. **Supervision:** OTP-style trees for automatic restart. **Timers:** Hierarchical wheels for O(1) operations.

4 Evaluation

Performance: 435K processes/sec creation, 1.2M msgs/sec, 85ns context switch. Supports millions of processes in reasonable memory.

Production: Powers systems with 100K+ connections, P99 <10ms. Supervision achieves 99.99% uptime.

vs Others: Akka (JVM threads)—heavyweight. Cloud Haskell—monadic. Go—no selective receive. Rust—async annotations everywhere. RIOT—lightweight, direct-style, complete actor semantics.

5 Related Work

Actor systems [1, 3] typically compromise key aspects. Effect handlers [2, 4] haven’t targeted actors. RIOT uniquely combines both.

6 Conclusion

Effect handlers transform language-level concurrency. RIOT proves OCaml can match purpose-built VMs while maintaining type safety. The key: effect handlers hit the sweet spot—avoiding manual continuations while enabling fine control.

<https://github.com/riot-ml/riot>

References

- [1] Joe Armstrong. Making reliable distributed systems in the presence of software errors. Phd dissertation, Royal Institute of Technology, Stockholm, 2003.
- [2] Daan Leijen. Koka: Programming with row polymorphic effect types. In *Proceedings 5th Workshop on Mathematically Structured Functional Programming*, MSFP 2014, pages 100–126, 2014.

- [3] Lightbend Inc. Akka: Build powerful reactive, concurrent, and distributed applications more easily. <https://akka.io>, 2023.
- [4] KC Sivaramakrishnan, Stephen Dolan, Leo White, Tom Kelly, Sadiq Jaffer, and Anil Madhavapeddy. Retrofitting effect handlers onto ocaml.

In *Proceedings of the 42nd ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '21*, pages 206–221. ACM, 2021.